BRICKHUB INTELLIGENCE

Coding-agent implementation brief | Pune residential micro-market ranking | Synthetic-data prototype (v1)

Purpose: Give this entire document to a coding agent and instruct it to implement the complete, runnable prototype, automated tests and submission artifacts. This plan is based on the supplied three-page Brickhub Intelligence take-home assignment dated 12 October 2026.

Critical data integrity rule: The assignment requires independently sourced public data, citation/date for every data point, and identification of estimates. The initial prototype requested here intentionally uses synthetic (dummy) data. All screens, files, charts and exported reports must prominently say “SYNTHETIC DEMO DATA - NOT INVESTMENT ADVICE / NOT VALID ASSIGNMENT EVIDENCE”. A synthetic-only version is useful for development but does not fully meet the assignment’s real-data sourcing requirement. Build easy replacement with documented public data before submission.

1. Scope and product definition

Parameter	Implementation requirement
City and coverage	Pune only; 12 residential micro-markets (configurable to 10-15).
Asset class	Residential apartments only; exclude land, commercial, industrial, villas and financial financing models.
Investor persona	Buy-and-hold for 5-7 years; cares about both appreciation potential and rental income.
Primary outcome	Rank all selected micro-markets; recommend top three with explainable score drivers and caveats.
Mandatory analytical elements	Method rationale; explicit missing-data treatment; ranking sensitivity/stability; honest limitations.
Tech recommendation	Python 3.11+, Streamlit, Pandas, NumPy, Plotly, pytest; open-source local execution.

Chosen micro-markets (synthetic coverage)

Baner, Wakad, Kharadi, Hadapsar, Hinjewadi, Balewadi, Bavdhan, Aundh, Wagholi, Pimple Saudagar, Kothrud and Undri. Add short, descriptive coverage rationale in the design note: geographic spread, mature versus emerging markets, residential / employment-corridor diversity. Clearly label this selection as judgmental, not a statistical sample.

2. User experience: required Streamlit application

Page / navigation	Required functionality
Overview dashboard	Demo warning banner; investor assumptions; coverage and last-run metadata; top-three recommendation cards; bar chart of all locality scores; visible methodological caveats.
Data explorer	Raw synthetic table; source_type/synthetic labels; data dictionary; missingness heatmap; filters; CSV download; edit/replace upload.
Method & weights	Explain indicators, direction, normalization, formula and weights; adjustable weight sliders totaling 100%; presets: Balanced, Yield-focused, Growth-focused; reset.
Market ranking	Sortable table (rank, score, evidence completeness, key drivers); radar/bar comparison for selected areas; explain each of top 3.
Robustness lab	Base vs scenario ranks; perturb weights; imputation strategies; leave-one-indicator-out; top-3 persistence frequency and rank-range visual.
Reports / exports	Download CSVs, chart PNGs if supported, method note, design note, analysis PDF; include demo disclaimers and run config.

UX rules: All filters, sliders, uploads, downloads, and scenario buttons must work. Show empty/error states and deterministic defaults. No fake “live market data” claims. Prioritize analysis transparency over visual complexity.

3. Synthetic dataset contract

Generate synthetic data using a fixed random seed (42) and stable locality-specific baselines. Use plausible demo ranges for Pune, but explicitly state they are fabricated examples, not observed market facts. Never assign a real online source to fabricated values.

Column	Type / rule
market_id, locality, city, asset_class	Unique ID; 12 distinct names; Pune; apartment.
median_price_inr_sqft	Positive numeric; synthetic price levels.
price_growth_3y_pct	Numeric; explicitly historical synthetic growth, not forecast.
gross_rental_yield_pct	Positive numeric; annual rent / current property value x 100.
annual_sales_transactions	Non-negative integer; demand proxy, not verified government registrations.
unsold_inventory_to_annual_sales	Positive numeric; higher = less favorable supply pressure.
infrastructure_score_0_100	Synthetic ordinal proxy; acknowledge subjective construction.
risk_score_0_100	Synthetic severity measure; higher risk = worse.
source_type, source_name, source_url, observation_date	For synthetic rows: source_type=synthetic; source_name=generator; URL blank; observation date is generation date, not observed market date.
is_estimated, is_synthetic, missing_reason	Explicit booleans and missingness tags; never conceal imputation.

Use a long-format provenance file with one record per market-indicator value (market_id, indicator, value, unit, observation_date, source_type, source_name, source_url, is_estimated, is_synthetic, missing_reason). A wide-format dataset may be generated from it for computations. Provide a clean public-data CSV template with the exact same provenance schema.

Synthetic missingness scenarios

• Create a complete underlying synthetic truth dataset, then a separate observed demo dataset with ~10-20% values missing across different indicators and markets (MCAR and patterned missingness).

• Inject a few deliberately questionable values/outliers into a separate QA fixture; validate and flag them, rather than silently replacing or deleting them.

• Offer three missing-data modes: median imputation; conservative/worst-reasonable imputation; complete-case scoring with explicit market eligibility / coverage rules. Keep imputed flags and before/after output.

4. Scoring methodology (explicit, auditable and reproducible)

Default method: indicator-wise min-max scaling to 0-100 across selected markets. For favorable indicators, score = 100 x (x - min)/(max - min); for unfavorable indicators, score = 100 x (max - x)/(max - min). If min=max, assign all valid values 50 and flag no discriminatory power. Use fixed comparable bounds or document that relative sample bounds shift with added markets.

Indicator / direction	Default weight	Meaning
Synthetic price growth / higher better	25%	Historical appreciation proxy; no future guarantee.
Gross rental yield / higher better	20%	Income proxy; gross of vacancies, tax, costs.
Annual sales / higher better	20%	Demand / liquidity proxy, affected by market size.
Infrastructure / higher better	15%	Accessibility / services composite demo score.
Inventory-to-sales / lower better	10%	Oversupply pressure proxy.
Risk severity / lower better	10%	Investment downside proxy.

Composite score = sum(indicator_normalized_score x weight_decimal). Validate nonnegative weights and sum = 1.0. Ranked order sorted by descending score, stable alphabetical tie-break. Output weighted contributions for each market and a market-level evidence-completeness metric. Median price remains a displayed contextual metric; if used in scoring, require a separate explicit affordability rationale (do not reward low price blindly).

Worked-example checks

• Create a unit test where all indicators are fixed, so manually calculated weights produce an exact known score.

• Verify reversing a risk indicator causes higher risk to reduce the score; changing weights reorders results and top-3 cards consistently.

• Never use target leakage, projected returns, or future known outcomes to compute score. These rankings are heuristic decision support, not causal investment forecasts.

5. Robustness and missing-data experiments

Test	Implementation	Output
Weight sensitivity	Base model plus Yield-focused (e.g., 15/35/20/10/10/10) and Growth-focused (40/10/20/15/5/10), and >= 500 seeded random valid weight vectors around base.	Per-market top-3 frequency, median rank, 10th-90th percentile rank, rank volatility.
Imputation sensitivity	Run median vs conservative vs complete-case with an eligibility threshold; handle low coverage explicitly.	Base/change rank and top-3 turnover; list largest movers.
Leave-one-out	Drop one indicator at a time; renormalize remaining weights.	Compare rank correlations and areas entering/leaving top 3.
Data perturbation	For numeric measures add documented plausible synthetic noise with seed; >= 200 trials.	Top-3 stability distribution and cross-run movement.
Validation	Tests for missingness, invalid fields, consistent units, stable seeds, no future-date fabrication and determinism.	Printed QA results and test pass summary.

Never claim the top-three ranking is “robust” without a declared numerical criterion. Suggested: call a market stable when it appears in the top 3 in >= 80% of prespecified scenarios; label this threshold as project-defined, not a Brickhub requirement. Show the actual numerator/denominator and separate weight vs imputation sensitivity.

6. Architecture and proposed repository

Pipeline: source CSV/generator -> Pydantic/Pandas validation -> cleaning + provenance preservation -> scoring engine -> scenario/stability engine -> visualization and report generation -> Streamlit app and downloadable artifacts. All calculations must live in pure, testable modules (not embedded in UI callbacks).

- app.py - Streamlit navigation, session state, controls, downloads.
- src/generate_demo.py - deterministic synthetic long-form provenance and wide demo export.
- src/schema.py / validation.py - schemas, units, constraints, data-quality report.
- src/preprocessing.py - missing-value strategies, outlier flags, transformations.
- src/scoring.py - normalization, weighted scores, ranking, contributions.
- src/sensitivity.py - preset weights, random vectors, uncertainty perturbations, leave-one-out.
- src/reports.py / plots.py - charts and generated downloadable reports.
- config/indicators.yaml - indicator direction, weight, units, acceptable ranges, fixed seed.
- data/demo_*.csv; data/public_data_template.csv - generated demo only and empty live-data template.
- tests/ - unit, integration and smoke tests; README.md, requirements.txt, run scripts.

7. Source-compliant deliverables from the application

Requested assignment artifact	What the coding agent must generate
1. Separate design note (1-2 pages)	Pre-build methodology, investor decision, selection logic, indicators, scoring choice, missingness plan, anticipated limitations. Save and timestamp before coding. Do not retroactively rewrite as though it were pre-build.
2. Reproducible prototype + method note (<= 1 page)	Runnable code from generated raw input to ranking; data provenance; cleaning, scaling, weights, missingness, tests; single-command execution.
3. Analysis & recommendation (1-2 pages)	Synthetic-demo top 3 and defensible *conditional* rationale; charts; rank stability; missing-data impact; evidence limitations and next steps.
4. 15-minute walkthrough	Speaker notes/demo script (not a substitute for the required live call), with timeboxes, one understandable trade-off and anticipated Q&A.;

Reporting disclosure: Every synthetic report must carry a watermark/header “DEMO - SYNTHETIC DATA” and a note that actual investment recommendations require replacement with public sourced records. If PDFs are generated on demand, include selected weights, missing strategy, data hash/seed and run timestamp for reproducibility.

8. Implementation phases and completion criteria

Phase	Coding agent activities	Acceptance gate
0. Record design	Create design_note.pdf first and freeze it as a pre-build artifact.	Design decision and date clearly stated.
1. Data layer	Create reproducible 12-market seed generator, provenance, missingness, public-data import template.	Same seed -> byte/record consistent business values; missingness visible.
2. Analytics	Build validators, missing strategies, scoring and explanation outputs.	Known-answer tests pass; all ranks/scores reproducible.
3. Robustness	Implement at least 3 weight models, 500 weight trials, 200 perturbation trials, leave-one-out.	Ranks and membership statistics export correctly.
4. Streamlit app	Implement all six functional screens, charts, upload and downloads.	No inert controls; all scenario UI states update.
5. Report and QA	Generate notes/reports and walkthrough, run pytest and smoke validation.	Artifact lengths valid; app starts; README exact commands.

9. Exact acceptance checklist for coding agent

- All 12 selected Pune markets appear consistently in dataset, app and CSV exports; city and asset class are fixed.
- Initial demo generator runs offline with seed 42 and no API keys or network dependency.
- There is a global and per-export synthetic-data disclaimer; no fabricated source attribution.
- Raw and derived fields, data dictionary, units and provenance can be inspected.
- Real/public-data import via documented CSV schema works without rewriting scoring code.
- Method choice and assumption justification are visible; weighted scoring is not a black box.
- Three missing-data modes work, keep flags and quantify changed scores/ranks.
- Three weight strategies and randomized stability tests work and show top-3 frequencies.

- The top-three recommendation is accompanied by score breakdown and uncertainty notes.
- Limitations explicitly reject guaranteed returns and individual-property valuation claims.
- Four assignment deliverables are represented, with the walkthrough script marked preparation for a live call.
- pytest succeeds; no runtime exceptions; no placeholder TODO, mock buttons, or screenshots pretending to be functioning features.

10. Paste-ready master instruction for the coding agent

Role: You are a senior Python data scientist and Streamlit engineer. Implement a finished, fully working, end-to-end Brickhub Intelligence Pune Residential Micro-market Ranking prototype using the attached implementation brief as the source of truth. Do not stop at UI mockups or pseudocode. Generate the complete project, synthetic CSVs, validation, scoring modules, sensitivity engine, tests, docs and exported PDF reports.

Execution: First write and freeze the 1-2-page design note. Then implement phases 1-5. Use Python 3.11+, Streamlit, Pandas, NumPy, Plotly, pytest and an offline seed-42 generator with 12 markets. Add a CSV upload path for eventual public data and preserve per-observation provenance. Display a prominent synthetic-data warning everywhere. Implement weighted scoring, three missingness strategies, three weight presets, random sensitivity, leave-one-out analysis, stability metrics, top-three recommendations, explanations and limitations. Deliver full editable source, requirements, README, tests and generated outputs.

Definition of done: Supply exact environment setup and start commands; show file tree; run pytest; perform smoke tests on calculations, scenario controls, uploaded CSV validation and downloads; report test results and open known limitations. Do not claim that synthetic demo outputs fulfill Brickhub’s public-data evidence requirement. No production financial forecasting, IRR or loan modeling.

11. Explicit limitations and real-data replacement plan

- Synthetic ranks prove software behavior, not real investment suitability. The eventual submission should replace synthetic indicators with publicly gathered observations, sources and observation dates, then rerun and discuss changed rankings.
- Historical price changes and gross rental yield do not establish future appreciation or net investor return; rent vacancies, taxes, fees and transaction frictions are excluded.
- Aggregate localities conceal micro-location and building-level differences. Sales counts confound liquidity with market size, and reported rental listings may be advertised rather than transacted rents.
- Infrastructure and risk composites are subjective; limited sample size (12 markets), normalization, missingness and choice of weighting can materially affect outcomes.
- No additional city, asset class, IRR, loan structuring or claims of validated forecasts. The final live walkthrough is scheduled by Brickhub after receipt of the submission.

Assignment reference: Brickhub Intelligence - Assignment (Anurag), dated 12 October 2026, pages 1-3. Deadline stated in original brief: end of day Friday, 16 October 2026. The weight profiles, code structure, metrics and UI specifications in this plan are implementation recommendations, not verbatim employer requirements.